

Get MPU6050 Data (I2C)

Get MPU6050 Data (I2C)

1. Learning Objectives
 - I2C Introduction
 - I2C Basic Parameters
2. Hardware Setup
3. Experimental Steps
 1. Open SYSCONFIG Configuration Tool
 2. Pin Parameter Configuration
 3. Serial Port Configuration
 4. I2C Protocol Usage
 5. Write Program
 6. Compile
4. Program Analysis
5. Experimental Phenomenon

1. Learning Objectives

1. Learn and understand basic I2C communication knowledge.
2. Get MPU6050 data.

I2C Introduction

IIC bus is a bidirectional two-wire synchronous bus that provides communication lines between integrated circuits. Its meaning is the protocol for exchanging information between integrated circuits or functional units.

The IIC module receives and transmits data, converting data from serial to parallel or parallel to serial. Interrupts can be enabled or disabled. The interface connects to the IIC bus through the data pin (SDA) and clock pin (SCL). It allows connection to standard (up to 100kHz) or fast (up to 400kHz) IIC buses. (Data line SDA and clock SCL constitute the serial bus, can send and receive data).

There are three types of signals during IIC bus data transmission: start signal (START), stop signal (STOP), and acknowledge signal (ACK). Secondly, it is in an idle state when no data transmission is in progress.

I2C Basic Parameters

Speed: I2C bus has two transmission modes: standard mode (100 kbit/s) and fast mode (400 kbit/s). There are also faster extended mode and high-speed mode available.

Device Address: Each device has a unique 7-bit or 10-bit address. Communication target is determined by address selection.

Bus Status: I2C bus has five states: idle state, start signal, stop signal, response signal, and data transmission.

Data Format: I2C bus has two data formats, standard format and fast format. Standard format is 8-bit data byte plus 1-bit ack/nack (acknowledge/not acknowledge) bit. Fast format allows two bytes to be transmitted simultaneously.

Since SCL and SDA lines are bidirectional, they may have level errors due to external reasons (such as capacitance in the circuit), which may cause communication errors. Therefore, in IIC bus, pull-up resistors are usually used to ensure the signal line is at high level in the idle state.

2. Hardware Setup

MSPM0G series I2C supports master-slave mode, has 7 address bits that can be set, supports 100kbps, 400kbps, 1Mbps I2C standard transmission rates, and supports SMBUS. Whether as master or slave, both sending and receiving have independent 8-byte FIFOs. MSPM0 I2C has 8-byte FIFO, generates independent interrupts for controller and target modes, and supports DMA.

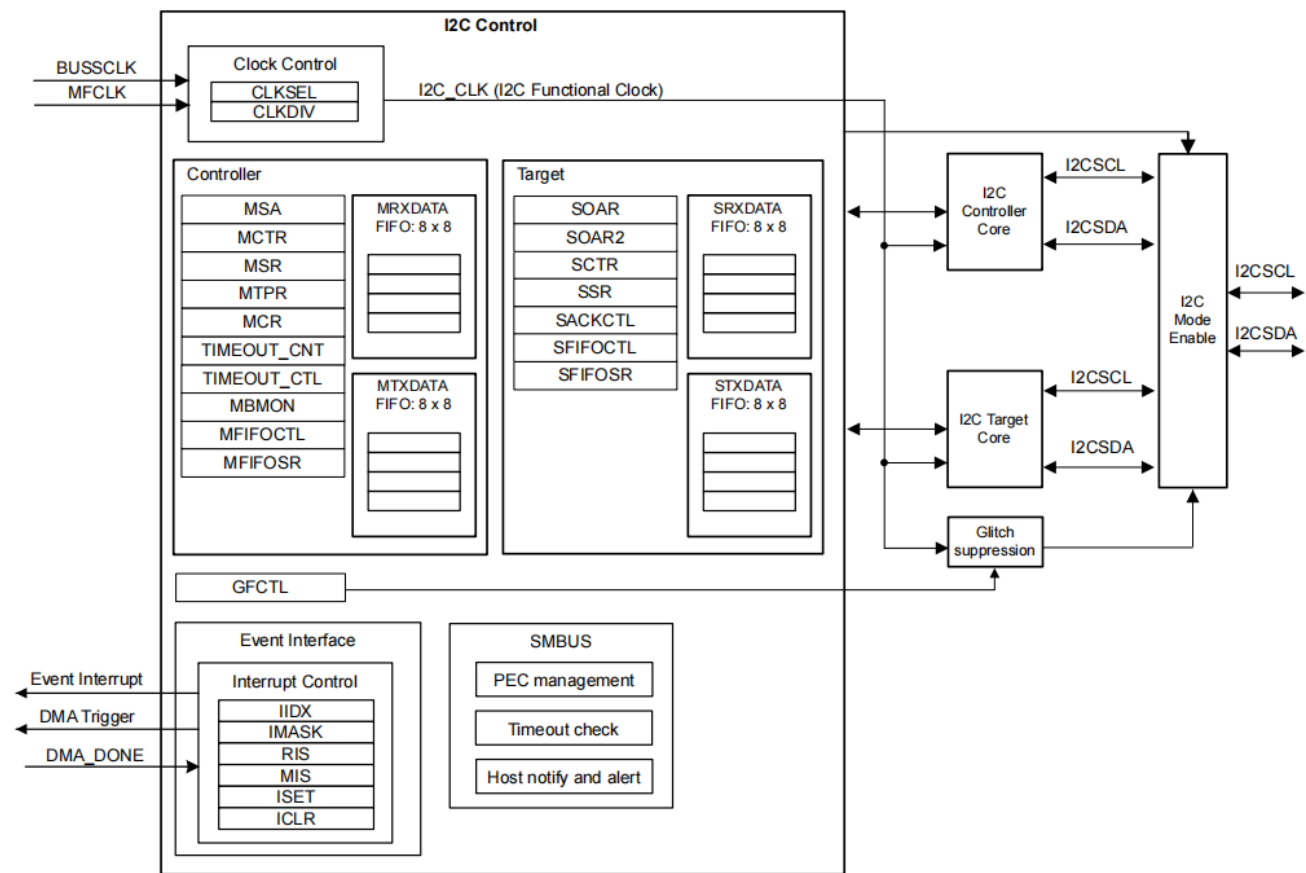


Figure 18-1. I2C Functional Block Diagram

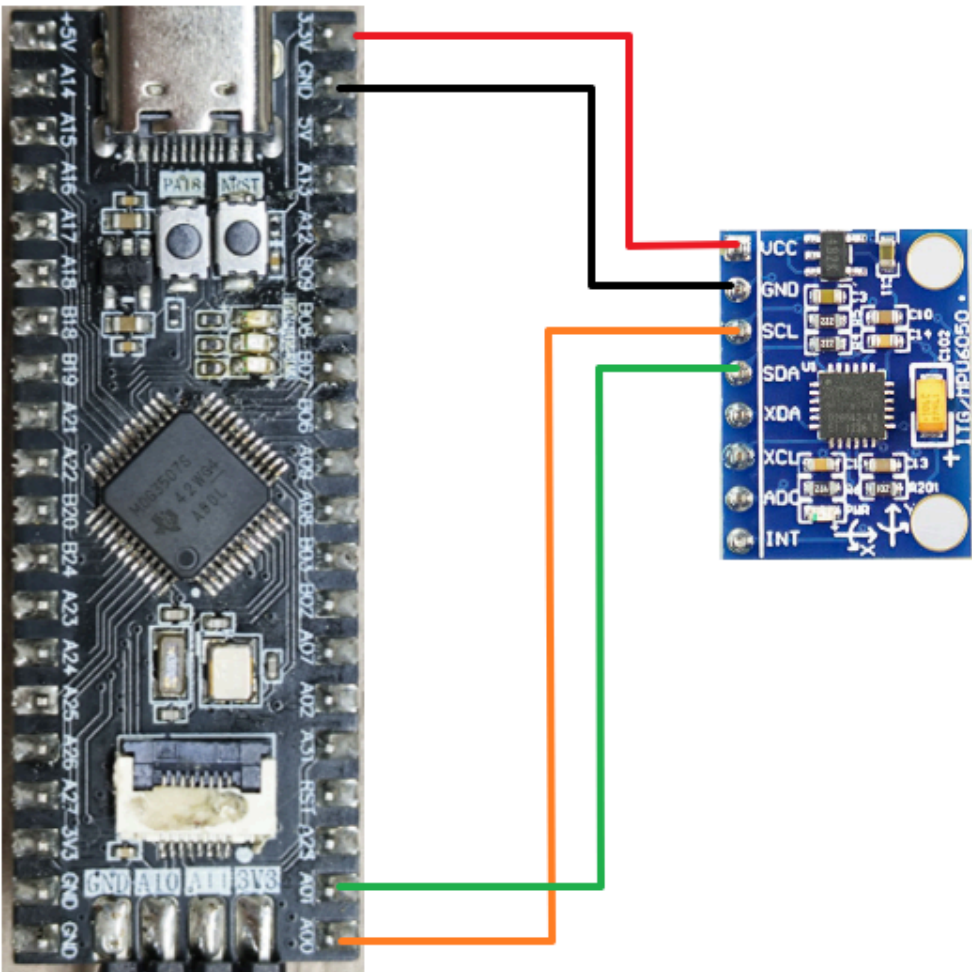
Software I2C refers to implementing I2C communication protocol by writing code in the program. It uses general-purpose input/output (GPIO) pins to simulate I2C data line (SDA) and clock line (SCL), and controls pin level changes via software to transmit data and generate timing signals. Compared with hardware I2C, the advantage of software I2C is that it does not require specific hardware support and can be implemented on any microcontroller that supports GPIO functionality. It uses the microcontroller's general-purpose IO pins to implement the I2C communication protocol.

Hardware I2C refers to handling I2C communication protocol through dedicated hardware modules. Most modern microcontrollers and some external devices have integrated hardware I2C modules. These hardware modules are responsible for handling I2C communication details, including generating correct timing signals, automatically handling signal conflicts, data transmission, and error detection. You can directly use hardware pins for connection without writing timing code.

This experiment uses software IIC to read MPU6050 module data.

Hardware Connection

MSPM0G3519	MPU6050
PA0	SCL
PA1	SDA
3V3	VCC
GND	GND



3. Experimental Steps

This course configures PA0 and PA1 pins as SCL and SDA to read data from the MPU6050 six-axis sensor module.

Here we use the template project we provided for introduction.

Place the template project in the SDK root directory and extract it. I renamed it as the 10_I2C folder.

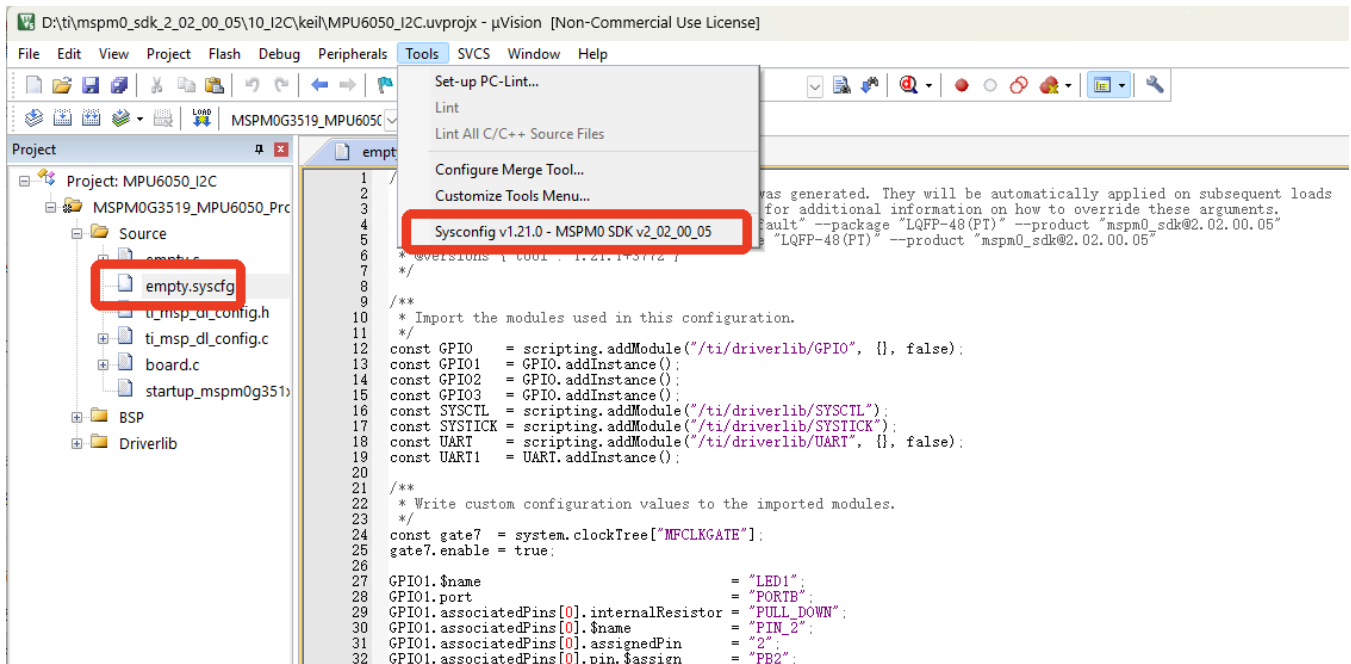
:) > ti > mspm0_sdk_2_02_00_05				在 mspm...
名称	修改日期	类型	大小	
01_LED	2024/11/19 11:42	文件夹		
02_Delay	2024/11/19 18:29	文件夹		
03_KEY	2024/11/19 20:09	文件夹		
04_ExtInterrupt	2024/11/19 21:15	文件夹		
05_UART	2024/11/20 19:41	文件夹		
06_Timer	2024/11/20 20:28	文件夹		
07_PWM	2024/11/21 15:59	文件夹		
08_ADC	2024/11/21 18:37	文件夹		
09_DMA	2024/11/21 19:48	文件夹		
10_I2C	2024/11/25 21:03	文件夹		
docs	2024/11/14 15:05	文件夹		
examples	2024/11/14 15:05	文件夹		

1. Open SYSCONFIG Configuration Tool

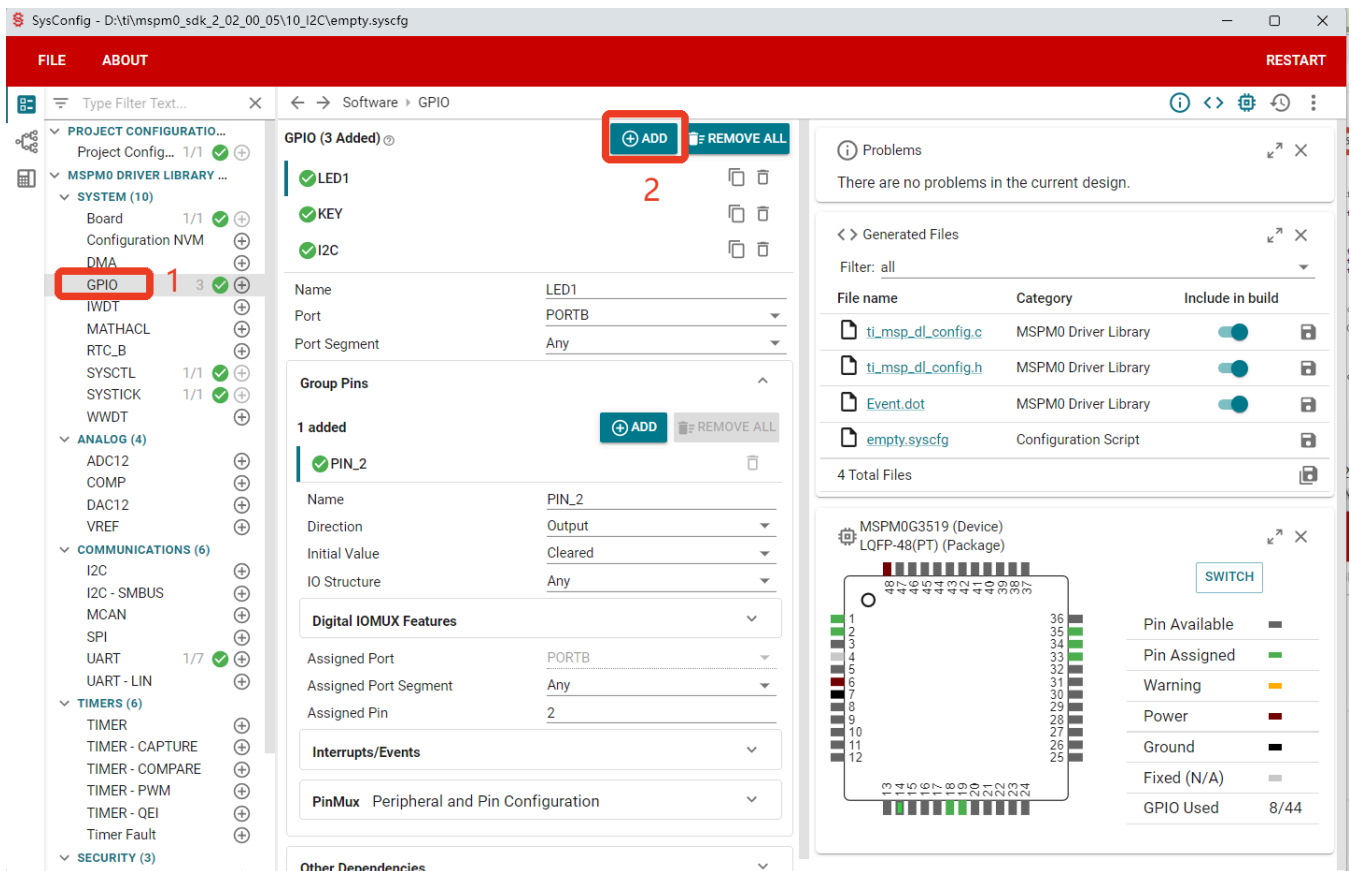
Then open the project.

新加卷 (D:) > ti > mspm0_sdk_2_02_00_05 > 10_I2C > keil				
Name	Date modified	Type	Size	
Listings	2026/9/9 18:04	File folder		
Objects	2026/9/9 18:04	File folder		
JLinkLog.txt	2026/8/18 18:55	Text Document	3 KB	
JLinkSettings.ini	2026/8/18 18:55	Configuration sett...	1 KB	
MPU6050_I2C.uvguix.Administrator	2026/8/19 18:31	ADMINISTRATOR ...	90 KB	
MPU6050_I2C.uvoptx	2026/8/19 18:30	UVOPTX File	18 KB	
MPU6050_I2C.uvprojx	2026/8/19 18:30	磳ision5 Project	20 KB	
mspm0g3519.sct	2024/1/25 11:42	Windows Script C...	1 KB	
startup_mspm0g351x_uvision.s	2026/8/19 18:26	S File	10 KB	

Open the empty.syscfg file in Keil's main interface. With the empty.syscfg file open, then open the SYSCONFIG GUI interface.



Find the GPIO section on the left, click to enter, and add a group of GPIOs.



2. Pin Parameter Configuration

SCL:

GPIO (3 Added) ⓘ

+ ADD

🗑️ REMOVE ALL

✓ LED1

✓ KEY

✓ I2C

📄

🗑️

📄

🗑️

📄

🗑️

Name

I2C

Port

Any

▼

Port Segment

Any

▼

Group Pins

^

1 added

+ ADD

🗑️ REMOVE ALL

✓ SCL

🗑️

Name

SCL

Direction

Output

▼

Initial Value

Set

▼

IO Structure

5V Tolerant Open Drain

▼

Digital IOMUX Features

▼

Assigned Port

PORTA

▼

Assigned Port Segment

Any

▼

Assigned Pin

0

Interrupts/Events

▼

LaunchPad-Specific Pin

No Shortcut Used

▼

PinMux Peripheral and Pin Configuration

^

SCL

PA0/33

▼

🔒

SDA:

GPIO (3 Added)

+ ADD
REMOVE ALL

LED1

KEY

I2C

Name
I2C

Port
Any

Port Segment
Any

Group Pins
2 added

+ ADD
REMOVE ALL

SCL

SDA

Name
SDA

Direction
Output

Initial Value
Set

IO Structure
5V Tolerant Open Drain

Digital IOMUX Features

Assigned Port
PORTA

Assigned Port Segment
Any

Assigned Pin
1

Interrupts/Events

LaunchPad-Specific Pin
No Shortcut Used

PinMux Peripheral and Pin Configuration

SDA
Any(PA1/34)

Other Dependencies

3. Serial Port Configuration

This course uses I2C protocol to send MPU6050 data to the host computer (computer) via serial port for display. So we also need to configure the serial port.

The unshown part is the default option.

- ▼ PROJECT CONFIGURATION...
 - Project Config... 1/1 ✓ ⊕
- ▼ MSPM0 DRIVER LIBRARY ...
- ▼ SYSTEM (9)
 - Board 1/1 ✓ ⊕
 - Configuration NVM ⊕
 - DMA ⊕
 - GPIO 3 ✓ ⊕
 - MATHACL ⊕
 - RTC ⊕
 - SYSCTL 1/1 ✓ ⊕
 - SYSTICK 1/1 ✓ ⊕
 - WWDT ⊕
- ▼ ANALOG (6)
 - ADC12 ⊕
 - COMP ⊕
 - DAC12 ⊕
 - GPAMP ⊕
 - OPA ⊕
 - VREF ⊕
- ▼ COMMUNICATIONS (6)
 - I2C ⊕
 - I2C - SMBUS ⊕
 - MCAN ⊕
 - SPI ⊕
 - UART 1/4 ✓ ⊕
 - UART - LIN ⊕

UART (1 of 4 Added) ⓘ

⊕ ADD

🗑 REMOVE ALL

✓ UART_0



Name	UART_0
Selected Peripheral	UART0

Quick Profiles ^

UART Profiles Custom ▾

Basic Configuration ^

UART Initialization Configuration ^

Clock Source	MFCLK ▾
Clock Divider	Divide by 1 ▾
Calculated Clock Source	4.00 MHz
Target Baud Rate	9600
Calculated Baud Rate	9598.08 ▾
Calculated Error (%)	0.02
Word Length	8 bits ▾
Parity	None ▾
Stop Bits	One ▾
HW Flow Control	Disable HW flow control ▾

Advanced Configuration

UART Mode

Normal UART Mode

Communication Direction

TX and RX

Oversampling

16x

Enable FIFOs

☐

Analog Glitch Filter

Disabled

Digital Glitch Filter

0

Calculated Digital Glitch Filter

0.00 s

RX Timeout Interrupt Counts

0

Calculated RX Timeout Interrupt

0.00 s

Enable Internal Loopback

☐

Enable Majority Voting

☐

Enable MSB First

☐

Retention Configuration

Low-Power Register Retention

Registers retained

Disable Retention APIs

☐

Extend Configuration

Interrupt Configuration

Enable Interrupts

Receive

Interrupt Priority

Default

PinMux

Peripheral and Pin Configuration

UART Peripheral

UART0

RX Pin

PA11/57

TX Pin

PA10/56

4. I2C Protocol Usage

We create two new files in the project template, namely bsp_mpu6050.c and bsp_mpu6050.h. Save them in the BSP folder of the project.

加卷 (D:) > ti > mspm0_sdk_2_02_00_05 > 10_I2C > BSP >

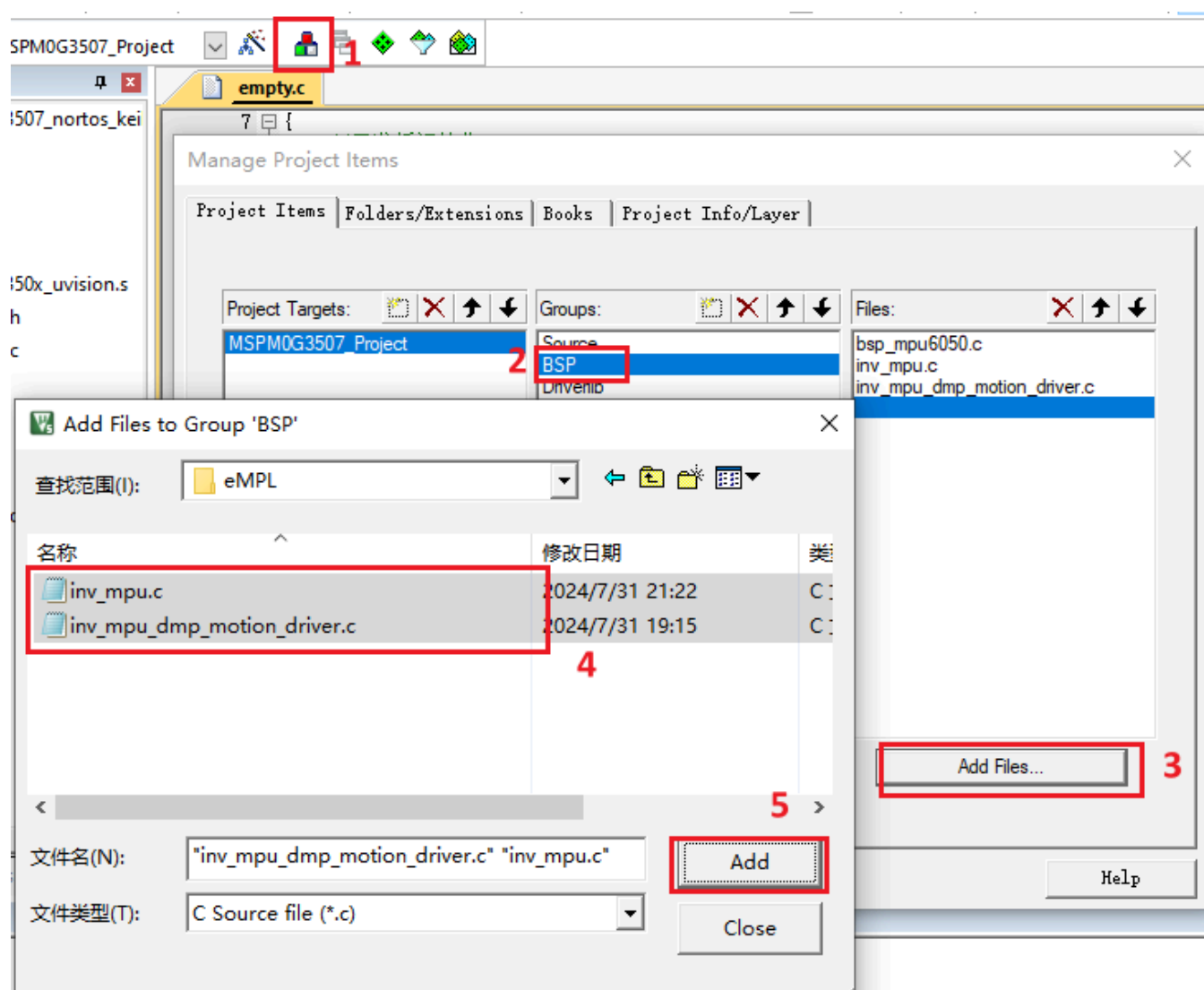
名称	修改日期	类型	大小
 bsp_mpu6050.h	2024/11/26 10:38	C Header 源文件	4 KB
 bsp_mpu6050.c	2024/11/26 10:16	C 文件	13 KB

Include the following MPU6050 gyroscope and accelerometer driver design files, placed in the eMPL folder

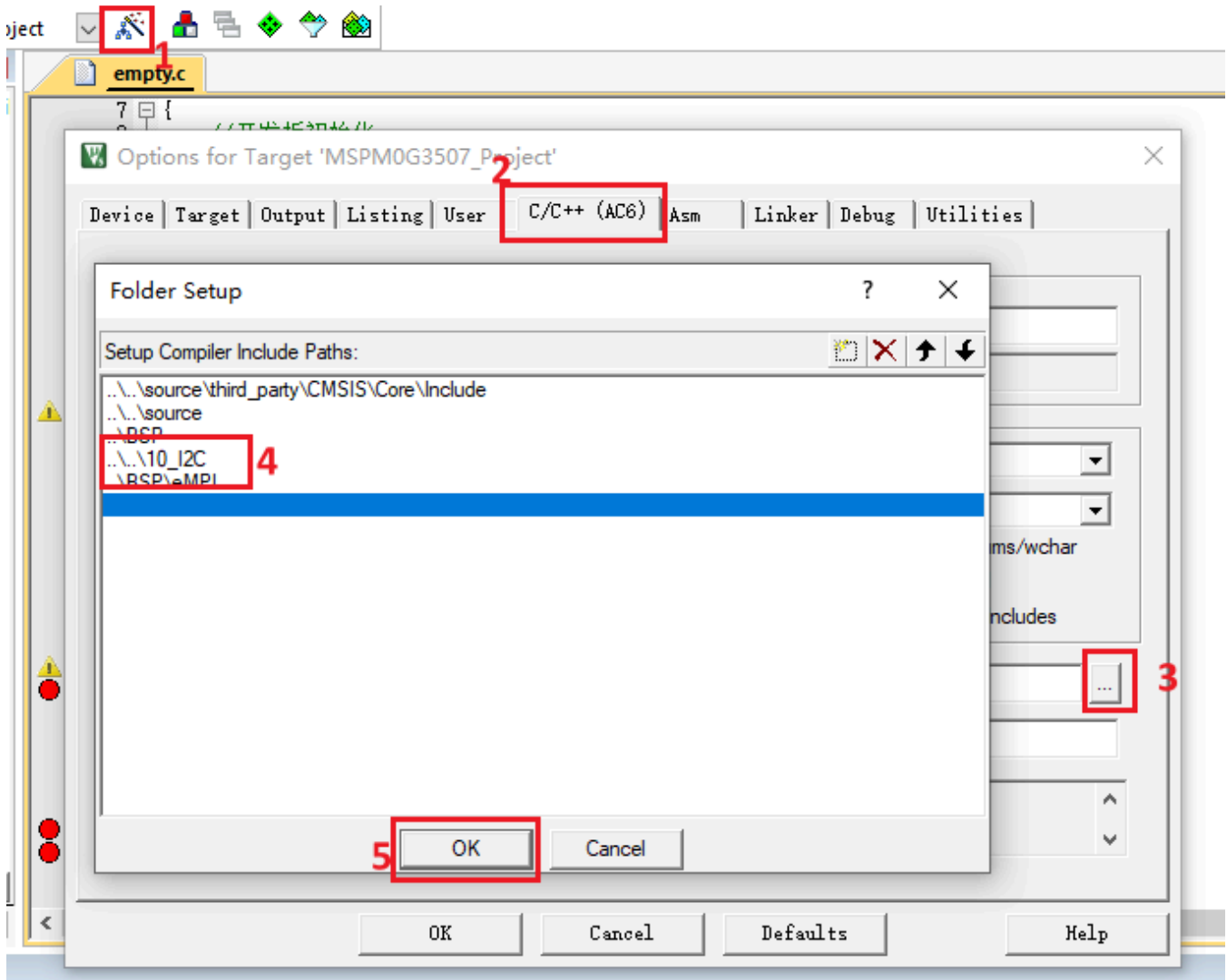
加卷 (D:) > ti > mspm0_sdk_2_02_00_05 > 10_I2C > BSP > eMPL

名称	修改日期	类型	大小
dmpKey.h	2024/7/31 19:15	C Header 源文件	19 KB
dmpmap.h	2024/7/31 19:15	C Header 源文件	7 KB
inv_mpu.c	2024/7/31 21:22	C 文件	87 KB
inv_mpu.h	2024/7/31 19:15	C Header 源文件	5 KB
inv_mpu_dmp_motion_driver.c	2024/7/31 19:15	C 文件	57 KB
inv_mpu_dmp_motion_driver.h	2024/7/31 19:15	C Header 源文件	4 KB

Open project management, click BSP, and add the files we previously created and copied to BSP.



Update header file paths.



5. Write Program

When using software to implement I2C communication, you need to select appropriate pins for data line (SDA) and clock line (SCL). Usually, any programmable GPIO pin can be selected as the software I2C pin. For software I2C, you need at least two pins for data line (SDA) and clock line (SCL), and ensure these pins can meet the timing requirements of I2C communication protocol. The following is the general pin description:

1. Data line (SDA): Pin for transmitting data. In software I2C, you need to set this pin to output mode (for master device sending data) and input mode (for master device receiving data). During communication, you need to control data line level changes to achieve data transmission.
2. Clock line (SCL): Pin for controlling data transmission clock signal. In software I2C, you need to set this pin to output mode, and control clock line level changes to generate clock pulses to control data line transmission. Note that when selecting appropriate pins, consider the following aspects:
 - Support input/output configuration: Pins need to support configuring as input or output mode in software, and be able to dynamically switch via program.
 - Hardware limitations and conflicts: Ensure the selected pins are not assigned to other hardware functions or peripherals to avoid conflicts.
 - Electrical characteristics: Pin electrical characteristics should meet I2C bus standard requirements, such as correct voltage levels and drive capability.

Note that software I2C implementation requires more program code and computation. Compared with hardware I2C, software I2C is more sensitive to processor performance and timing control. Therefore, when selecting pins, you also need to consider processor performance and programmable performance. To ensure code maintainability and portability, related functions are macro-defined here.

SDA pin and SCL pin macro definitions are as follows:

bsp_mpu6050.h

```
#ifndef _BSP_MPU6050_H_
#define _BSP_MPU6050_H_

#include "board.h"

//Set SDA output mode
#define SDA_OUT() { \
    DL_GPIO_initDigitalOutput(I2C_SDA_IOMUX); \
    DL_GPIO_setPins(I2C_PORT, I2C_SDA_PIN); \
    DL_GPIO_enableOutput(I2C_PORT, I2C_SDA_PIN); \
}

//Set SDA input mode
#define SDA_IN() { DL_GPIO_initDigitalInput(I2C_SDA_IOMUX); }

//Get the level change of the SDA pin
#define SDA_GET() ( ( ( DL_GPIO_readPins(I2C_PORT,I2C_SDA_PIN) & I2C_SDA_PIN ) > 0 ) ? 1 : 0 )

//SDA and SCL output
#define SDA(x) ( (x) ? (DL_GPIO_setPins(I2C_PORT,I2C_SDA_PIN)) : (DL_GPIO_clearPins(I2C_PORT,I2C_SDA_PIN)) )
#define SCL(x) ( (x) ? (DL_GPIO_setPins(I2C_PORT,I2C_SCL_PIN)) : (DL_GPIO_clearPins(I2C_PORT,I2C_SCL_PIN)) )

//AD0 of MPU6050 is the IIC address pin. If it is grounded, the IIC address is 0x68. If it is connected to VCC, the IIC address is 0x69.

#define MPU6050_RA_SMPLRT_DIV    0x19    //Gyro sampling rate address
#define MPU6050_RA_CONFIG        0x1A    //Set digital low-pass filter address
#define MPU6050_RA_GYRO_CONFIG   0x1B    //Gyro configuration register
#define MPU6050_RA_ACCEL_CONFIG   0x1C    //Acceleration sensor configuration register
#define MPU_INT_EN_REG           0x38    //Interrupt enable register
#define MPU_USER_CTRL_REG        0x6A    //User control register
#define MPU_FIFO_EN_REG          0x23    //FIFO enable register
#define MPU_PWR_MGMT2_REG        0x6C    //Power management register 2
#define MPU_GYRO_CFG_REG         0x1B    //Gyroscope configuration register
#define MPU_ACCEL_CFG_REG        0x1C    //Accelerometer configuration register
#define MPU_CFG_REG              0x1A    //Configuration register
#define MPU_SAMPLE_RATE_REG      0x19    //Sampling frequency divider
#define MPU_INTBP_CFG_REG        0x37    //Interrupt/bypass setting register

#define MPU6050_RA_PWR_MGMT_1    0x6B
```

```

#define MPU6050_RA_PWR_MGMT_2      0x6C

#define MPU6050_WHO_AM_I           0x75
#define MPU6050_SMPLRT_DIV         0           //8000Hz
#define MPU6050_DLPF_CFG           0
#define MPU6050_GYRO_OUT           0x43       //MPU6050 gyroscope data register address
#define MPU6050_ACC_OUT            0x3B       //MPU6050 acceleration data register
address

#define MPU6050_RA_TEMP_OUT_H       0x41       //High temperature
#define MPU6050_RA_TEMP_OUT_L       0x42       //Low temperature

#define MPU_ACCEL_XOUTH_REG          0x3B       //Acceleration value, X-axis high 8-bit
register
#define MPU_ACCEL_XOUTL_REG          0x3C       //Acceleration value, X-axis low 8-bit
register
#define MPU_ACCEL_YOUTH_REG          0x3D       //Acceleration value, Y-axis high 8-bit
register
#define MPU_ACCEL_YOUTL_REG          0x3E       //Acceleration value, Y-axis low 8-bit
register
#define MPU_ACCEL_ZOUTH_REG          0x3F       //Acceleration value, Z-axis high 8-bit
register
#define MPU_ACCEL_ZOUTL_REG          0x40       //Acceleration value, Z-axis low 8-bit
register

#define MPU_TEMP_OUTH_REG            0x41       //Temperature value high eight bits register
#define MPU_TEMP_OUTL_REG            0x42       //Temperature value lower 8 bits register

#define MPU_GYRO_XOUTH_REG            0x43       //Gyroscope value, X-axis high 8-bit
register
#define MPU_GYRO_XOUTL_REG            0x44       //Gyroscope value, X-axis low 8-bit register
#define MPU_GYRO_YOUTH_REG            0x45       //Gyroscope value, Y-axis high 8-bit
register
#define MPU_GYRO_YOUTL_REG            0x46       //Gyroscope value, Y-axis low 8-bit register
#define MPU_GYRO_ZOUTH_REG            0x47       //Gyroscope value, Z-axis high 8-bit
register
#define MPU_GYRO_ZOUTL_REG            0x48       //Gyroscope value, Z-axis low 8-bit register

char MPU6050_WriteReg(uint8_t addr,uint8_t regaddr,uint8_t num,uint8_t *regdata);
char MPU6050_ReadData(uint8_t addr, uint8_t regaddr,uint8_t num,uint8_t* Read);

char MPU6050_Init(void);
void MPU6050ReadGyro(short *gyroData);
void MPU6050ReadAcc(short *accData);
float MPU6050_GetTemp(void);
uint8_t MPU6050ReadID(void);
#endif

```

Next is configuring the I2C timing part,

bsp_mpu6050.c (only partial content is shown here, see project source code for details)

```

#include "bsp_mpu6050.h"
#include "stdio.h"

/*****
 * Function name: IIC_Start
 * Function description: IIC start timing
 * Function parameter: None
 * Function return: None
 * Author: LC
 * Remarks: None
 *****/
void IIC_Start(void)
{
    SDA_OUT();
    SCL(1);
    SDA(0);

    SDA(1);
    delay_us(5);
    SDA(0);
    delay_us(5);

    SCL(0);
}

/*****
 * Function name: IIC_Stop
 * Function description: IIC stop signal
 * Function parameters: None
 * Function return: None
 * Author: LC
 * Notes: None
 *****/
void IIC_Stop(void)
{
    SDA_OUT();
    SCL(0);
    SDA(0);

    SCL(1);
    delay_us(5);
    SDA(1);
    delay_us(5);
}

...

```

Then in the empty.c file, write the following code

```

#include "board.h"
#include <stdio.h>
#include "bsp_mpu6050.h"
#include "inv_mpu.h"

```

```

int main(void)
{
    //Development board initialization
    board_init();

    MPU6050_Init();

    uint8_t ret = 1;

    float pitch=0,roll=0,yaw=0;    //Euler Angles

    printf("start\r\n");

    //DMP Initialization
    while( mpu_dmp_init() )
    {
        printf("dmp error\r\n");
        delay_ms(200);
    }

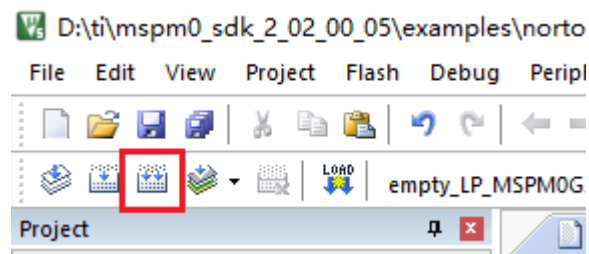
    printf("Initialization Data Succeed \r\n");

    while(1)
    {
        //Get Euler angles
        if( mpu_dmp_get_data(&pitch,&roll,&yaw) == 0 )
        {
            printf("\r\npitch =%d\r\n", (int)pitch);
            printf("\r\nroll =%d\r\n", (int)roll);
            printf("\r\nyaw =%d\r\n", (int)yaw);
        }
        delay_ms(200); //According to the set sampling rate, the delay cannot be set too
        large
    }
}

```

6. Compile

Click the Rebuild icon. The following prompt indicates compilation is complete with no errors.



Build Output

```
compiling dl_factoryregion.c...
compiling dl_timer.c...
compiling dl_uart.c...
compiling dl_mcan.c...
linking...
Program Size: Code=17944 RO-data=4008 RW-data=56 ZI-data=592
FromELF: creating hex file...
".\Objects\MPU6050_I2C.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:02
```

4. Program Analysis

- inv_mpu.c

```
//mpu6050, dmp initialization
//Return value: 0, normal
//          other, failed
u8 mpu_dmp_init(void)
{
    u8 res=0;

    res = mpu_init();
    //    printf("res = %d\r\n",res);
    if(res==0) //Initialize MPU6050
    {
        res=mpu_set_sensors(INV_XYZ_GYRO|INV_XYZ_ACCEL);//Set up the required sensors
        if(res)return 1;
        res=mpu_configure_fifo(INV_XYZ_GYRO | INV_XYZ_ACCEL);//Set up FIFO
        if(res)return 2;
        res=mpu_set_sample_rate(DEFAULT_MPU_HZ);    //Set the Sample Rate
        if(res)return 3;
        res=dmp_load_motion_driver_firmware();    //Load dmp firmware
        if(res)return 4;
        res=dmp_set_orientation(inv_orientation_matrix_to_scalar(gyro_orientation));//Set
gyroscope orientation
        if(res)return 5;
        res=dmp_enable_feature(DMP_FEATURE_6X_LP_QUAT|DMP_FEATURE_TAP| //Set the dmp
function
        DMP_FEATURE_ANDROID_ORIENT|DMP_FEATURE_SEND_RAW_ACCEL|DMP_FEATURE_SEND_CAL_GYRO|
        DMP_FEATURE_GYRO_CAL);
        if(res)return 6;
        res=dmp_set_fifo_rate(DEFAULT_MPU_HZ); //Set the DMP output rate (maximum 200Hz)
        if(res)return 7;
    //    res=run_self_test();    //Self-Test
    //    if(res)return 8;
        res=mpu_set_dmp_state(1); //Enable DMP
        if(res)return 9;
    }

    return 0;
}
```

This code implements MPU6050 initialization, providing efficient motion tracking and attitude calculation capability through DMP. Call `mpu_init` to initialize the hardware module; enable gyroscope and accelerometer and configure their sample rates and FIFO cache; load DMP firmware to support advanced functions (such as attitude calculation and gesture detection); set device orientation matrix and DMP feature settings (such as quaternion calculation, data calibration, etc.); finally enable DMP output, allowing the device to provide stable attitude and sensor data at the set frequency.

```
//Get the data after dmp processing (note that this function requires a lot of stacks and a
lot of local variables)
//pitch: pitch angle    accuracy: 0.1°    range: -90.0° <---> +90.0°
//roll: roll angle      accuracy: 0.1°    range: -180.0° <---> +180.0°
//yaw: heading angle    accuracy: 0.1°    range: -180.0° <---> +180.0°
//Return value: 0, normal
//                others, failed
u8 mpu_dmp_get_data(float *pitch, float *roll, float *yaw)
{
    float q0=1.0f, q1=0.0f, q2=0.0f, q3=0.0f;
    unsigned long sensor_timestamp;
    short gyro[3], accel[3], sensors;
    unsigned char more;
    long quat[4];
    if(dmp_read_fifo(gyro, accel, quat, &sensor_timestamp, &sensors, &more)) return 1;
    /* Gyro and accel data are written to the FIFO by the DMP in chip frame and hardware
units.
    * This behavior is convenient because it keeps the gyro and accel outputs of
dmp_read_fifo and mpu_read_fifo consistent.
    */
    /*if (sensors & INV_XYZ_GYRO )
send_packet(PACKET_TYPE_GYRO, gyro);
if (sensors & INV_XYZ_ACCEL)
send_packet(PACKET_TYPE_ACCEL, accel); */
    /* Unlike gyro and accel, quaternions are written to the FIFO in the body frame, q30.
    * The orientation is set by the scalar passed to dmp_set_orientation during
initialization.
    */
    if(sensors & INV_WXYZ_QUAT)
    {
        q0 = quat[0] / q30; //Convert q30 format to floating point number
        q1 = quat[1] / q30;
        q2 = quat[2] / q30;
        q3 = quat[3] / q30;
        //Calculate the pitch angle/roll angle/heading angle
        *pitch = asin(-2 * q1 * q3 + 2 * q0 * q2) * 57.3; // pitch
        *roll  = atan2(2 * q2 * q3 + 2 * q0 * q1, -2 * q1 * q1 - 2 * q2 * q2 + 1) * 57.3; //
roll
        *yaw   = atan2(2 * (q1 * q2 + q0 * q3), q0 * q0 + q1 * q1 - q2 * q2 - q3 * q3) * 57.3; //yaw
    } else return 2;
    return 0;
}
```

This function reads DMP-processed sensor data (including gyroscope, accelerometer, and quaternion) from MPU6050's FIFO, calculates pitch, roll, and yaw by parsing quaternions, and returns the results.

- empty.c

```
int main(void)
{
    //Development board initialization
    board_init();

    MPU6050_Init();

    uint8_t ret = 1;

    float pitch=0,roll=0,yaw=0;    //Euler Angles

    printf("start\r\n");

    //DMP Initialization
    while( mpu_dmp_init() )
    {
        printf("dmp error\r\n");
        delay_ms(200);
    }

    printf("Initialization Data Succeed \r\n");

    while(1)
    {
        //Get Euler angles
        if( mpu_dmp_get_data(&pitch,&roll,&yaw) == 0 )
        {
            printf("\r\npitch =%d\r\n", (int)pitch);
            printf("\r\nroll =%d\r\n", (int)roll);
            printf("\r\nyaw =%d\r\n", (int)yaw);
        }
        delay_ms(200); //According to the set sampling rate, the delay cannot be set too
        large
    }
}
```

This program implements real-time attitude angle measurement of MPU6050. By initializing the development board and MPU6050 and loading the DMP module, the device's pitch angle, roll angle, and yaw angle are calculated. The program first performs DMP initialization, retries on failure, and after success enters the main loop to continuously read attitude angle data and output via serial port.

5. Experimental Phenomenon

After the program is downloaded, configure the serial assistant as shown in the figure below. Open the serial port to read real-time data from the MPU6050 module.

